JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Class DecimalFormat

java.lang.Object
    java.text.Format
        java.text.NumberFormat
            java.text.DecimalFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

```
public class DecimalFormat
extends NumberFormat
```

`DecimalFormat` is a concrete subclass of `NumberFormat` that formats decimal numbers. It has a variety of features designed to make it possible to parse and format numbers in any locale, including support for Western, Arabic, and Indic digits. It also supports different kinds of numbers, including integers (123), fixed-point numbers (123.4), scientific notation (1.23E4), percentages (12%), and currency amounts ($123). All of these can be localized.

To obtain a `NumberFormat` for a specific locale, including the default locale, call one of `NumberFormat`'s factory methods, such as `getInstance()`. In general, do not call the `DecimalFormat` constructors directly, since the `NumberFormat` factory methods may return subclasses other than `DecimalFormat`. If you need to customize the format object, do something like this:

```
NumberFormat f = NumberFormat.getInstance(loc);
if (f instanceof DecimalFormat) {
    ((DecimalFormat) f).setDecimalSeparatorAlwaysShown(true);
}
```

A `DecimalFormat` comprises a *pattern* and a set of *symbols*. The pattern may be set directly using `applyPattern()`, or indirectly using the API methods. The symbols are stored in a `DecimalFormatSymbols` object. When using the `NumberFormat`

factory methods, the pattern and symbols are read from localized `ResourceBundle`s.

Patterns

`DecimalFormat` patterns have the following syntax:

Pattern:
PositivePattern
PositivePattern ; *NegativePattern*
PositivePattern:
Prefix$_{opt}$ *Number* *Suffix*$_{opt}$
NegativePattern:
Prefix$_{opt}$ *Number* *Suffix*$_{opt}$
Prefix:
any Unicode characters except \uFFFE, \uFFFF, and special characters
Suffix:
any Unicode characters except \uFFFE, \uFFFF, and special characters
Number:
Integer *Exponent*$_{opt}$
Integer . *Fraction* *Exponent*$_{opt}$
Integer:
MinimumInteger
#
Integer
, *Integer*
MinimumInteger:
0
0 *MinimumInteger*
0 , *MinimumInteger*
Fraction:
MinimumFraction$_{opt}$ *OptionalFraction*$_{opt}$
MinimumFraction:
0 *MinimumFraction*$_{opt}$
OptionalFraction:
OptionalFraction$_{opt}$
Exponent:
E *MinimumExponent*
MinimumExponent:
0 *MinimumExponent*$_{opt}$

A `DecimalFormat` pattern contains a positive and negative subpattern, for example, `"#,##0.00;(#,##0.00)"`. Each subpattern has a prefix, numeric part, and suffix. The negative subpattern is optional; if absent, then the positive subpattern prefixed with the localized minus sign (`'-'` in most locales) is used as the negative subpattern. That is, `"0.00"` alone is equivalent to `"0.00;-0.00"`. If there is an explicit negative subpattern, it serves only to specify the negative

prefix and suffix; the number of digits, minimal digits, and other characteristics are all the same as the positive pattern. That means that `"#,##0.0#;(#)"` produces precisely the same behavior as `"#,##0.0#;(#,##0.0#)"`.

The prefixes, suffixes, and various symbols used for infinity, digits, thousands separators, decimal separators, etc. may be set to arbitrary values, and they will appear properly during formatting. However, care must be taken that the symbols and strings do not conflict, or parsing will be unreliable. For example, either the positive and negative prefixes or the suffixes must be distinct for `DecimalFormat.parse()` to be able to distinguish positive from negative values. (If they are identical, then `DecimalFormat` will behave as if no negative subpattern was specified.) Another example is that the decimal separator and thousands separator should be distinct characters, or parsing will be impossible.

The grouping separator is commonly used for thousands, but in some countries it separates ten-thousands. The grouping size is a constant number of digits between the grouping characters, such as 3 for 100,000,000 or 4 for 1,0000,0000. If you supply a pattern with multiple grouping characters, the interval between the last one and the end of the integer is the one that is used. So `"#,##,###,####" == "######,####" == "##,####,####"`.

Special Pattern Characters

Many characters in a pattern are taken literally; they are matched during parsing and output unchanged during formatting. Special characters, on the other hand, stand for other characters, strings, or classes of characters. They must be quoted, unless noted otherwise, if they are to appear in the prefix or suffix as literals.

The characters listed here are used in non-localized patterns. Localized patterns use the corresponding characters taken from this formatter's `DecimalFormatSymbols` object instead, and these characters lose their special status. Two exceptions are the currency sign and quote, which are not localized.

Symbol	Location	Localized?	Meaning
`0`	Number	Yes	Digit
`#`	Number	Yes	Digit, zero shows as absent
`.`	Number	Yes	Decimal separator or monetary decimal separator
`-`	Number	Yes	Minus sign
`,`	Number	Yes	Grouping separator
`E`	Number	Yes	Separates mantissa and exponent in scientific notation. *Need not be quoted in prefix or suffix.*
`;`	Subpattern boundary	Yes	Separates positive and negative subpatterns
`%`	Prefix or suffix	Yes	Multiply by 100 and show as percentage

`\u2030`	Prefix or suffix	Yes	Multiply by 1000 and show as per mille value
¤ (\u00A4)	Prefix or suffix	No	Currency sign, replaced by currency symbol. If doubled, replaced by international currency symbol. If present in a pattern, the monetary decimal separator is used instead of the decimal separator.
'	Prefix or suffix	No	Used to quote special characters in a prefix or suffix, for example, `"'#'#"` formats 123 to `"#123"`. To create a single quote itself, use two in a row: `"# o''clock"`.

Scientific Notation

Numbers in scientific notation are expressed as the product of a mantissa and a power of ten, for example, 1234 can be expressed as 1.234 x 10^3. The mantissa is often in the range 1.0 ≤ x < 10.0, but it need not be. `DecimalFormat` can be instructed to format and parse scientific notation *only via a pattern*; there is currently no factory method that creates a scientific notation format. In a pattern, the exponent character immediately followed by one or more digit characters indicates scientific notation. Example: `"0.###E0"` formats the number 1234 as `"1.234E3"`.

- The number of digit characters after the exponent character gives the minimum exponent digit count. There is no maximum. Negative exponents are formatted using the localized minus sign, *not* the prefix and suffix from the pattern. This allows patterns such as `"0.###E0 m/s"`.
- The minimum and maximum number of integer digits are interpreted together:
    - If the maximum number of integer digits is greater than their minimum number and greater than 1, it forces the exponent to be a multiple of the maximum number of integer digits, and the minimum number of integer digits to be interpreted as 1. The most common use of this is to generate *engineering notation*, in which the exponent is a multiple of three, e.g., `"##0.#####E0"`. Using this pattern, the number 12345 formats to `"12.345E3"`, and 123456 formats to `"123.456E3"`.
    - Otherwise, the minimum number of integer digits is achieved by adjusting the exponent. Example: 0.00123 formatted with `"00.###E0"` yields `"12.3E-4"`.
- The number of significant digits in the mantissa is the sum of the *minimum integer* and *maximum fraction* digits, and is unaffected by the maximum integer digits. For example, 12345 formatted with `"##0.##E0"` is `"12.3E3"`. To show all digits, set the significant digits count to zero. The number of significant digits does not affect parsing.
- Exponential patterns may not contain grouping separators.

Rounding

`DecimalFormat` provides rounding modes defined in `RoundingMode` for formatting. By default, it uses `RoundingMode.HALF_EVEN`.

Digits

For formatting, `DecimalFormat` uses the ten consecutive characters starting with the localized zero digit defined in the `DecimalFormatSymbols` object as digits. For parsing, these digits as well as all Unicode decimal digits, as defined by `Character.digit`, are recognized.

Special Values

`NaN` is formatted as a string, which typically has a single character `\uFFFD`. This string is determined by the `DecimalFormatSymbols` object. This is the only value for which the prefixes and suffixes are not used.

Infinity is formatted as a string, which typically has a single character `\u221E`, with the positive or negative prefixes and suffixes applied. The infinity string is determined by the `DecimalFormatSymbols` object.

Negative zero (`"-0"`) parses to

- `BigDecimal(0)` if `isParseBigDecimal()` is true,
- `Long(0)` if `isParseBigDecimal()` is false and `isParseIntegerOnly()` is true,
- `Double(-0.0)` if both `isParseBigDecimal()` and `isParseIntegerOnly()` are false.

Synchronization

Decimal formats are generally not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

Example

```
<strong>// Print out a number using the localized number, integer, currency,
// and percent format for each locale</strong>
Locale[] locales = NumberFormat.getAvailableLocales();
double myNumber = -1234.56;
NumberFormat form;
for (int j = 0; j < 4; ++j) {
    System.out.println("FORMAT");
    for (int i = 0; i < locales.length; ++i) {
        if (locales[i].getCountry().length() == 0) {
           continue; // Skip language-only locales
        }
        System.out.print(locales[i].getDisplayName());
        switch (j) {
```

```
            case 0:
                form = NumberFormat.getInstance(locales[i]); break;
            case 1:
                form = NumberFormat.getIntegerInstance(locales[i]); break;
            case 2:
                form = NumberFormat.getCurrencyInstance(locales[i]); break;
            default:
                form = NumberFormat.getPercentInstance(locales[i]); break;
            }
            if (form instanceof DecimalFormat) {
                System.out.print(": " + ((DecimalFormat) form).toPattern());
            }
            System.out.print(" -> " + form.format(myNumber));
            try {
                System.out.println(" -> " + form.parse(form.format(myNumber)));
            } catch (ParseException e) {}
        }
    }
```

See Also:

Java Tutorial, NumberFormat, DecimalFormatSymbols, ParsePosition, Serialized Form

Nested Class Summary

Nested classes/interfaces inherited from class java.text.NumberFormat

NumberFormat.Field

Field Summary

Fields inherited from class java.text.NumberFormat

FRACTION_FIELD, INTEGER_FIELD

Constructor Summary

Constructors

Constructor	Description
DecimalFormat()	

	Creates a DecimalFormat using the default pattern and symbols for the default **FORMAT** locale.
DecimalFormat(**String** pattern)	Creates a DecimalFormat using the given pattern and the symbols for the default **FORMAT** locale.
DecimalFormat(**String** pattern, **DecimalFormatSymbols** symbols)	Creates a DecimalFormat using the given pattern and symbols.

Method Summary

Concrete Methods

Modifier and Type	Method
void	**applyLocalizedPattern**(**String** pattern)
void	**applyPattern**(**String** pattern)
Object	**clone**()
boolean	**equals**(**Object** obj)
StringBuffer	**format**(double number, **StringBuffer** result, **FieldPosition** fieldPosition)
StringBuffer	**format**(long number, **StringBuffer** result, **FieldPosition** fieldPosition)
StringBuffer	**format**(**Object** number, **StringBuffer** toAppendTo, **FieldPosition** pos)

AttributedCharacterIterator	formatToCharacterIterator(Object obj)
Currency	getCurrency()
DecimalFormatSymbols	getDecimalFormatSymbols()
int	getGroupingSize()
int	getMaximumFractionDigits()
int	getMaximumIntegerDigits()
int	getMinimumFractionDigits()
int	getMinimumIntegerDigits()
int	getMultiplier()
String	getNegativePrefix()
String	getNegativeSuffix()
String	getPositivePrefix()
String	getPositiveSuffix()
RoundingMode	getRoundingMode()
int	hashCode()
boolean	isDecimalSeparatorAlwaysShown()

boolean	**isParseBigDecimal**()
Number	**parse**(**String** text, **ParsePosition** pos)
void	**setCurrency**(**Currency** currency)
void	**setDecimalFormatSymbols**(**DecimalFormatSymbols** newSymb
void	**setDecimalSeparatorAlwaysShown**(boolean newValue)
void	**setGroupingSize**(int newValue)
void	**setGroupingUsed**(boolean newValue)
void	**setMaximumFractionDigits**(int newValue)
void	**setMaximumIntegerDigits**(int newValue)
void	**setMinimumFractionDigits**(int newValue)
void	**setMinimumIntegerDigits**(int newValue)
void	**setMultiplier**(int newValue)

void	**setNegativePrefix**(**String** newValue)
void	**setNegativeSuffix**(**String** newValue)
void	**setParseBigDecimal**(boolean newValue)
void	**setPositivePrefix**(**String** newValue)
void	**setPositiveSuffix**(**String** newValue)
void	**setRoundingMode**(**RoundingMode** roundingMode)
String	**toLocalizedPattern**()
String	**toPattern**()

Methods inherited from class java.text.NumberFormat

format, format, getAvailableLocales, getCurrencyInstance, getCurrencyInstance, getInstance, getInstance, getIntegerInstance, getIntegerInstance, getNumberInstance, getNumberInstance, getPercentInstance, getPercentInstance, isGroupingUsed, isParseIntegerOnly, parse, parseObject, setParseIntegerOnly

Methods inherited from class java.text.Format

format, parseObject

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

DecimalFormat

```
public DecimalFormat()
```

Creates a DecimalFormat using the default pattern and symbols for the default `FORMAT` locale. This is a convenient way to obtain a DecimalFormat when internationalization is not the main concern.

To obtain standard formats for a given locale, use the factory methods on NumberFormat such as getNumberInstance. These factories will return the most appropriate sub-class of NumberFormat for a given locale.

See Also:

`NumberFormat.getInstance()`, `NumberFormat.getNumberInstance()`, `NumberFormat.getCurrencyInstance()`, `NumberFormat.getPercentInstance()`

DecimalFormat

```
public DecimalFormat(String pattern)
```

Creates a DecimalFormat using the given pattern and the symbols for the default `FORMAT` locale. This is a convenient way to obtain a DecimalFormat when internationalization is not the main concern.

To obtain standard formats for a given locale, use the factory methods on NumberFormat such as getNumberInstance. These factories will return the most appropriate sub-class of NumberFormat for a given locale.

Parameters:

`pattern - a non-localized pattern string.`

Throws:

`NullPointerException - if pattern is null`

`IllegalArgumentException - if the given pattern is invalid.`

See Also:

`NumberFormat.getInstance()`, `NumberFormat.getNumberInstance()`, `NumberFormat.getCurrencyInstance()`, `NumberFormat.getPercentInstance()`

DecimalFormat

```
public DecimalFormat(String pattern,
                     DecimalFormatSymbols symbols)
```

Creates a DecimalFormat using the given pattern and symbols. Use this constructor when you need to completely customize the behavior of the format.

To obtain standard formats for a given locale, use the factory methods on NumberFormat such as getInstance or getCurrencyInstance. If you need only minor adjustments to a standard format, you can modify the format returned by a NumberFormat factory method.

Parameters:

`pattern` - a non-localized pattern string

`symbols` - the set of symbols to be used

Throws:

`NullPointerException` - if any of the given arguments is null

`IllegalArgumentException` - if the given pattern is invalid

See Also:

`NumberFormat.getInstance()`, `NumberFormat.getNumberInstance()`, `NumberFormat.getCurrencyInstance()`, `NumberFormat.getPercentInstance()`, `DecimalFormatSymbols`

Method Detail

format

```
public final StringBuffer format(Object number,
                                 StringBuffer toAppendTo,
                                 FieldPosition pos)
```

Formats a number and appends the resulting text to the given string buffer. The number can be of any subclass of `Number`.

This implementation uses the maximum precision permitted.

Overrides:

`format` in class `NumberFormat`

Parameters:

`number` - the number to format

toAppendTo - the StringBuffer to which the formatted text is to be appended

pos - On input: an alignment field, if desired. On output: the offsets of the alignment field.

Returns:

the value passed in as toAppendTo

Throws:

IllegalArgumentException - if number is null or not an instance of Number.

NullPointerException - if toAppendTo or pos is null

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

FieldPosition

format

```
public StringBuffer format(double number,
                           StringBuffer result,
                           FieldPosition fieldPosition)
```

Formats a double to produce a string.

Specified by:

format in class NumberFormat

Parameters:

number - The double to format

result - where the text is to be appended

fieldPosition - On input: an alignment field, if desired. On output: the offsets of the alignment field.

Returns:

The formatted number string

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

FieldPosition

format

```
public StringBuffer format(long number,
                           StringBuffer result,
                           FieldPosition fieldPosition)
```

Format a long to produce a string.

Specified by:

format in class NumberFormat

Parameters:

number - The long to format

result - where the text is to be appended

fieldPosition - On input: an alignment field, if desired. On output: the offsets of the alignment field.

Returns:

The formatted number string

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

FieldPosition

formatToCharacterIterator

```
public AttributedCharacterIterator formatToCharacterIterator(Object obj)
```

Formats an Object producing an AttributedCharacterIterator. You can use the returned AttributedCharacterIterator to build the resulting String, as well as to determine information about the resulting String.

Each attribute key of the AttributedCharacterIterator will be of type NumberFormat.Field, with the attribute value being the same as the attribute key.

Overrides:

formatToCharacterIterator in class Format

Parameters:

obj - The object to format

Returns:

AttributedCharacterIterator describing the formatted value.

Throws:

NullPointerException - if obj is null.

IllegalArgumentException - when the Format cannot format the given object.

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

Since:

1.4

parse

```
public Number parse(String text,
                    ParsePosition pos)
```

Parses text from a string to produce a `Number`.

The method attempts to parse text starting at the index given by `pos`. If parsing succeeds, then the index of `pos` is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed number is returned. The updated `pos` can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of `pos` is not changed, the error index of `pos` is set to the index of the character where the error occurred, and null is returned.

The subclass returned depends on the value of `isParseBigDecimal()` as well as on the string being parsed.

- If `isParseBigDecimal()` is false (the default), most integer values are returned as `Long` objects, no matter how they are written: `"17"` and `"17.000"` both parse to `Long(17)`. Values that cannot fit into a `Long` are returned as `Doubles`. This includes values with a fractional part, infinite values, `NaN`, and the value -0.0. `DecimalFormat` does *not* decide whether to return a `Double` or a `Long` based on the presence of a decimal separator in the source string. Doing so would prevent integers that overflow the mantissa of a double, such as `"-9,223,372,036,854,775,808.00"`, from being parsed accurately.

  Callers may use the `Number` methods `doubleValue`, `longValue`, etc., to obtain the type they want.

- If `isParseBigDecimal()` is true, values are returned as `BigDecimal` objects. The values are the ones constructed by `BigDecimal.BigDecimal(String)` for corresponding strings in locale-independent format. The special cases negative and positive infinity and NaN are returned as `Double` instances holding the values of the corresponding `Double` constants.

`DecimalFormat` parses all Unicode characters that represent decimal digits, as defined by `Character.digit()`. In addition, `DecimalFormat` also

recognizes as digits the ten consecutive characters starting with the localized zero digit defined in the DecimalFormatSymbols object.

Specified by:

parse in class NumberFormat

Parameters:

text - the string to be parsed

pos - A ParsePosition object with index and error index information as described above.

Returns:

the parsed value, or null if the parse fails

Throws:

NullPointerException - if text or pos is null.

See Also:

NumberFormat.isParseIntegerOnly(), Format.parseObject(java.lang.String, java.text.ParsePosition)

getDecimalFormatSymbols

public DecimalFormatSymbols getDecimalFormatSymbols()

Returns a copy of the decimal format symbols, which is generally not changed by the programmer or user.

Returns:

a copy of the desired DecimalFormatSymbols

See Also:

DecimalFormatSymbols

setDecimalFormatSymbols

public void setDecimalFormatSymbols(DecimalFormatSymbols newSymbols)

Sets the decimal format symbols, which is generally not changed by the programmer or user.

Parameters:

newSymbols - desired DecimalFormatSymbols

See Also:

DecimalFormatSymbols

getPositivePrefix

```
public String getPositivePrefix()
```

Get the positive prefix.

Examples: +123, $123, sFr123

Returns:

the positive prefix

setPositivePrefix

```
public void setPositivePrefix(String newValue)
```

Set the positive prefix.

Examples: +123, $123, sFr123

Parameters:

newValue - the new positive prefix

getNegativePrefix

```
public String getNegativePrefix()
```

Get the negative prefix.

Examples: -123, ($123) (with negative suffix), sFr-123

Returns:

the negative prefix

setNegativePrefix

```
public void setNegativePrefix(String newValue)
```

Set the negative prefix.

Examples: -123, ($123) (with negative suffix), sFr-123

Parameters:

newValue - the new negative prefix

getPositiveSuffix

```
public String getPositiveSuffix()
```

Get the positive suffix.

Example: 123%

Returns:

the positive suffix

setPositiveSuffix

```
public void setPositiveSuffix(String newValue)
```

Set the positive suffix.

Example: 123%

Parameters:

newValue - the new positive suffix

getNegativeSuffix

```
public String getNegativeSuffix()
```

Get the negative suffix.

Examples: -123%, ($123) (with positive suffixes)

Returns:

the negative suffix

setNegativeSuffix

```
public void setNegativeSuffix(String newValue)
```

Set the negative suffix.

Examples: 123%

Parameters:

newValue - the new negative suffix

getMultiplier

```
public int getMultiplier()
```

Gets the multiplier for use in percent, per mille, and similar formats.

Returns:

the multiplier

See Also:

`setMultiplier(int)`

setMultiplier

```
public void setMultiplier(int newValue)
```

Sets the multiplier for use in percent, per mille, and similar formats. For a percent format, set the multiplier to 100 and the suffixes to have '%' (for Arabic, use the Arabic percent sign). For a per mille format, set the multiplier to 1000 and the suffixes to have '\u2030'.

Example: with multiplier 100, 1.23 is formatted as "123", and "123" is parsed into 1.23.

Parameters:

newValue - the new multiplier

See Also:

`getMultiplier()`

setGroupingUsed

```
public void setGroupingUsed(boolean newValue)
```

Set whether or not grouping will be used in this format.

Overrides:

`setGroupingUsed` in class `NumberFormat`

Parameters:

newValue - true if grouping is used; false otherwise

See Also:

`NumberFormat.isGroupingUsed()`

getGroupingSize

```
public int getGroupingSize()
```

Return the grouping size. Grouping size is the number of digits between grouping separators in the integer portion of a number. For example, in the number "123,456.78", the grouping size is 3.

Returns:

the grouping size

See Also:

`setGroupingSize(int)`, `NumberFormat.isGroupingUsed()`, `DecimalFormatSymbols.getGroupingSeparator()`

setGroupingSize

```
public void setGroupingSize(int newValue)
```

Set the grouping size. Grouping size is the number of digits between grouping separators in the integer portion of a number. For example, in the number "123,456.78", the grouping size is 3.
The value passed in is converted to a byte, which may lose information.

Parameters:

newValue - the new grouping size

See Also:

`getGroupingSize()`, `NumberFormat.setGroupingUsed(boolean)`, `DecimalFormatSymbols.setGroupingSeparator(char)`

isDecimalSeparatorAlwaysShown

```
public boolean isDecimalSeparatorAlwaysShown()
```

Allows you to get the behavior of the decimal separator with integers. (The decimal separator will always appear with decimals.)

Example: Decimal ON: 12345 → 12345.; OFF: 12345 → 12345

Returns:

true if the decimal separator is always shown; false otherwise

setDecimalSeparatorAlwaysShown

```
public void setDecimalSeparatorAlwaysShown(boolean newValue)
```

Allows you to set the behavior of the decimal separator with integers. (The decimal separator will always appear with decimals.)

Example: Decimal ON: 12345 → 12345.; OFF: 12345 → 12345

Parameters:

`newValue - true if the decimal separator is always shown; false otherwise`

isParseBigDecimal

```
public boolean isParseBigDecimal()
```

Returns whether the `parse(java.lang.String, java.text.ParsePosition)` method returns `BigDecimal`. The default value is false.

Returns:

`true if the parse method returns BigDecimal; false otherwise`

Since:

`1.5`

See Also:

`setParseBigDecimal(boolean)`

setParseBigDecimal

```
public void setParseBigDecimal(boolean newValue)
```

Sets whether the `parse(java.lang.String, java.text.ParsePosition)` method returns `BigDecimal`.

Parameters:

`newValue - true if the parse method returns BigDecimal; false otherwise`

Since:

`1.5`

See Also:

`isParseBigDecimal()`

clone

```
public Object clone()
```

Standard override; no change in semantics.

Overrides:

clone in class NumberFormat

Returns:

a clone of this instance.

See Also:

Cloneable

equals

```
public boolean equals(Object obj)
```

Overrides equals

Overrides:

equals in class NumberFormat

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is the same as the obj argument; false otherwise.

See Also:

Object.hashCode(), HashMap

hashCode

```
public int hashCode()
```

Overrides hashCode

Overrides:

hashCode in class NumberFormat

Returns:

a hash code value for this object.

See Also:

Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)

toPattern

```
public String toPattern()
```

Synthesizes a pattern string that represents the current state of this Format object.

Returns:

a pattern string

See Also:

applyPattern(java.lang.String)

toLocalizedPattern

```
public String toLocalizedPattern()
```

Synthesizes a localized pattern string that represents the current state of this Format object.

Returns:

a localized pattern string

See Also:

applyPattern(java.lang.String)

applyPattern

```
public void applyPattern(String pattern)
```

Apply the given pattern to this Format object. A pattern is a short-hand specification for the various formatting properties. These properties can also be changed individually through the various setter methods.

There is no limit to integer digits set by this routine, since that is the typical end-user desire; use setMaximumInteger if you want to set a real value. For negative numbers, use a second pattern, separated by a semicolon

Example `"#,#00.0#"` → 1,234.56

This means a minimum of 2 integer digits, 1 fraction digit, and a maximum of 2 fraction digits.

Example: `"#,#00.0#;(#,#00.0#)"` for negatives in parentheses.

In negative patterns, the minimum and maximum counts are ignored; these are presumed to be set in the positive pattern.

Parameters:

`pattern - a new pattern`

Throws:

`NullPointerException - if pattern is null`

`IllegalArgumentException - if the given pattern is invalid.`

applyLocalizedPattern

```
public void applyLocalizedPattern(String pattern)
```

Apply the given pattern to this Format object. The pattern is assumed to be in a localized notation. A pattern is a short-hand specification for the various formatting properties. These properties can also be changed individually through the various setter methods.

There is no limit to integer digits set by this routine, since that is the typical end-user desire; use setMaximumInteger if you want to set a real value. For negative numbers, use a second pattern, separated by a semicolon

Example `"#,#00.0#"` → 1,234.56

This means a minimum of 2 integer digits, 1 fraction digit, and a maximum of 2 fraction digits.

Example: `"#,#00.0#;(#,#00.0#)"` for negatives in parentheses.

In negative patterns, the minimum and maximum counts are ignored; these are presumed to be set in the positive pattern.

Parameters:

`pattern - a new pattern`

Throws:

`NullPointerException - if pattern is null`

`IllegalArgumentException - if the given pattern is invalid.`

setMaximumIntegerDigits

```
public void setMaximumIntegerDigits(int newValue)
```

Sets the maximum number of digits allowed in the integer portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of `newValue` and 309 is used. Negative input values are replaced with 0.

Overrides:

`setMaximumIntegerDigits` in class `NumberFormat`

Parameters:

`newValue - the maximum number of integer digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.`

See Also:

`NumberFormat.setMaximumIntegerDigits(int)`

setMinimumIntegerDigits

```
public void setMinimumIntegerDigits(int newValue)
```

Sets the minimum number of digits allowed in the integer portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of `newValue` and 309 is used. Negative input values are replaced with 0.

Overrides:

`setMinimumIntegerDigits` in class `NumberFormat`

Parameters:

`newValue - the minimum number of integer digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.`

See Also:

NumberFormat.setMinimumIntegerDigits(int)

setMaximumFractionDigits

```
public void setMaximumFractionDigits(int newValue)
```

Sets the maximum number of digits allowed in the fraction portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of `newValue` and 340 is used. Negative input values are replaced with 0.

Overrides:

`setMaximumFractionDigits` in class `NumberFormat`

Parameters:

`newValue` - the maximum number of fraction digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

NumberFormat.setMaximumFractionDigits(int)

setMinimumFractionDigits

```
public void setMinimumFractionDigits(int newValue)
```

Sets the minimum number of digits allowed in the fraction portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of `newValue` and 340 is used. Negative input values are replaced with 0.

Overrides:

`setMinimumFractionDigits` in class `NumberFormat`

Parameters:

`newValue` - the minimum number of fraction digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

NumberFormat.setMinimumFractionDigits(int)

getMaximumIntegerDigits

```
public int getMaximumIntegerDigits()
```

Gets the maximum number of digits allowed in the integer portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of the return value and 309 is used.

Overrides:

`getMaximumIntegerDigits` in class `NumberFormat`

Returns:

the maximum number of digits

See Also:

`setMaximumIntegerDigits(int)`

getMinimumIntegerDigits

```
public int getMinimumIntegerDigits()
```

Gets the minimum number of digits allowed in the integer portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of the return value and 309 is used.

Overrides:

`getMinimumIntegerDigits` in class `NumberFormat`

Returns:

the minimum number of digits

See Also:

`setMinimumIntegerDigits(int)`

getMaximumFractionDigits

```
public int getMaximumFractionDigits()
```

Gets the maximum number of digits allowed in the fraction portion of a number. For formatting numbers other than `BigInteger` and `BigDecimal` objects, the lower of the return value and 340 is used.

Overrides:

`getMaximumFractionDigits` in class `NumberFormat`

Returns:

the maximum number of digits.

See Also:

setMaximumFractionDigits(int)

getMinimumFractionDigits

public int getMinimumFractionDigits()

Gets the minimum number of digits allowed in the fraction portion of a number. For formatting numbers other than BigInteger and BigDecimal objects, the lower of the return value and 340 is used.

Overrides:

getMinimumFractionDigits in class NumberFormat

Returns:

the minimum number of digits

See Also:

setMinimumFractionDigits(int)

getCurrency

public Currency getCurrency()

Gets the currency used by this decimal format when formatting currency values. The currency is obtained by calling DecimalFormatSymbols.getCurrency on this number format's symbols.

Overrides:

getCurrency in class NumberFormat

Returns:

the currency used by this decimal format, or null

Since:

1.4

setCurrency

public void setCurrency(Currency currency)

Sets the currency used by this number format when formatting currency values. This does not update the minimum or maximum number of fraction digits used by the number format. The currency is set by calling DecimalFormatSymbols.setCurrency on this number format's symbols.

Overrides:

setCurrency in class NumberFormat

Parameters:

currency - the new currency to be used by this decimal format

Throws:

NullPointerException - if currency is null

Since:

1.4

getRoundingMode

```
public RoundingMode getRoundingMode()
```

Gets the RoundingMode used in this DecimalFormat.

Overrides:

getRoundingMode in class NumberFormat

Returns:

The RoundingMode used for this DecimalFormat.

Since:

1.6

See Also:

setRoundingMode(RoundingMode)

setRoundingMode

```
public void setRoundingMode(RoundingMode roundingMode)
```

Sets the RoundingMode used in this DecimalFormat.

Overrides:

setRoundingMode in class NumberFormat

Parameters:

roundingMode - The RoundingMode to be used

Throws:

NullPointerException - if roundingMode is null.

Since:

1.6

See Also:

```
getRoundingMode()
```

C

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.